

ArchitectAI Report

Project: Cloud Job Portal

Generated on: 14/6/2026

1. Executive Summary & Product Idea

I want to build a career portal with real-time chats

2. Structured Requirements

[Must Have] Feature based on Q1

Implementation details for question: "Do we need real-time chat?" with answer: "Yes, between employers and candidates"

[Must Have] Feature based on Q2

Implementation details for question: "Should we parsing resumes?" with answer: "Should support PDF imports"

[Must Have] User Management & Security

Secure registration, login, and token-based API authentication.

[Must Have] Responsive Dashboard UI

An interactive user dashboard displaying project metadata and items.

[Nice To Have] Data Archival & Retention

Automated periodic DB snapshots and user records cleanup tools.

3. Architecture Options

MERN Monolith (RECOMMENDED)

Rationale: Perfect for small-to-medium teams starting with a single codebase. A single Node.js app serves APIs and coordinates MongoDB connections directly.

Pros: Very fast initial development, Single source of truth, Simple hosting configurations

Cons: Tight coupling makes scaling individual modules difficult, Build times grow over time

Team Fit: Excellent fit for small teams with React/Node.js experience who need to deploy fast.

Scalability: Moderate. Scalable vertically or by running multiple node processes behind a load balancer.

Microservices

Rationale: Decoupled services using an API Gateway, with separate services for Auth, Projects, and AI processing.

Pros: High modularity, Independent scaling and deployment, Fault isolation

Cons: High operational overhead, Complex transaction boundaries, Complicated local setup

Team Fit: Suited for larger engineering organizations with dedicated DevOps capabilities.

Scalability: Excellent. Scale bottle-necked services (e.g., AI integration) independently.

Serverless

Rationale: Fully serverless deployment on AWS Lambda or Vercel, querying a hosted MongoDB database.

Pros: Scale to zero cost structure, No infrastructure management, Automatic provisioning

Cons: Cold starts, Vendor lock-in, Difficult local testing

Team Fit: Suited for teams that want low maintenance and have experience with serverless frameworks.

Scalability: Very high. Scales automatically with traffic bursts.

4. Technology Tradeoffs

MongoDB vs PostgreSQL

Scores - Cost: 4/5 | Scalability: 5/5 | Complexity: 3/5 | Performance: 4/5

Recommendation: MongoDB

Reasoning: MongoDB supports flexible schemas, perfect for storing varied architectural recommendations, decisions, and requirement shapes without rigid migrations.

REST vs GraphQL

Scores - Cost: 5/5 | Scalability: 4/5 | Complexity: 2/5 | Performance: 4/5

Recommendation: REST

Reasoning: REST is simpler and faster to build for standard CRUD operations on projects and decisions. We do not have deep nested relationships that warrant GraphQL overhead.

AWS vs Vercel

Scores - Cost: 3/5 | Scalability: 5/5 | Complexity: 4/5 | Performance: 5/5

Recommendation: Vercel

Reasoning: Vercel offers an unparalleled developer experience and seamless React integration for the frontend app, and can run our serverless functions easily.

5. Decision Memory Log

We will use MongoDB database with secondary index configurations

Decided by: Staff Architect | Date: 14/6/2026

Rationale: Auto-populated from Tradeoff Analysis: [Tradeoff: MongoDB vs PostgreSQL] Recommendation: MongoDB.

Reasoning: MongoDB supports flexible schemas, perfect for storing varied architectural recommendations, decisions, and requirement shapes without rigid migrations.

Rejected Alternatives: PostgreSQL